

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL X
Java Collections



Disusun Oleh:
Alifa Fatin Ramadhani 105224019

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

A. JavaCollections Framework

Java Collections Framework (JCF) adalah sekumpulan antarmuka (interface) dan kelas (class) standar di Java yang digunakan untuk menyimpan, memanipulasi, dan mencari sekumpulan objek secara dinamis. Berbeda dengan array tradisional yang ukurannya statis dan fiturnya terbatas, Collections menyediakan struktur data yang fleksibel, efisien, dan siap pakai.

B. Tiga Pilar Utama Java Collections

1. List (Daftar Terurut)

List adalah struktur data yang menyimpan elemen secara berurutan berdasarkan indeks dan mengizinkan elemen duplikat. Terdapat dua implementasi utama, yaitu ArrayList yang berbasis dynamic array dengan keunggulan pencarian cepat $O(1)$ berdasarkan indeks, dan LinkedList yang berbasis Doubly-Linked List dengan keunggulan penambahan atau penghapusan data di awal dan akhir daftar yang sangat cepat $O(1)$.

2. Set (Himpunan Unik)

Set adalah struktur data yang secara otomatis menolak elemen duplikat tanpa menghasilkan error. Terdapat dua implementasi utama, yaitu HashSet yang berbasis fungsi hash dengan pencarian elemen sangat cepat namun tidak menjamin urutan, dan TreeSet yang berbasis Red-Black Tree dengan elemen yang otomatis terurut secara ascending namun prosesnya sedikit lebih lambat dari HashSet.

3. Map (Pasangan Kunci-Nilai)

Map menyimpan data dalam bentuk Key-Value Pair, di mana Key bersifat unik sedangkan Value boleh duplikat. Implementasi yang umum digunakan adalah HashMap yang menyimpan data menggunakan hashing pada Key sehingga pencarian nilai berdasarkan Key diketahui berjalan sangat efisien.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
Class Barang			
1	idBarang	String	Menyimpan kode unik yang mengidentifikasi setiap barangnya
2	namaBarang	String	Menyimpan nama atau judul pada setiap barang
3	kategori	String	Menyimpan kategori atau kelompok barang (jenis barang)

4	stok	int	Menyimpan stok yang tersedia pada setiap barang
Class SistemGudang			
1	databaseBarang	Map<String, Barang>	Menyimpan semua objek barang dengan idBarang sebagai key yang memungkinkan proses pencarian data yang sangat cepat saat update stok
2	kategoriUnik	Set<String> kategoriUnik	Menyimpan daftar kategori barang tanpa duplikat, yang artinya setiap kategori hanya muncul satu kali
3	riwayat	LinkedList<String> riwayat	mencatat setiap aktivitas secara berurutan jika ada yang baru maka akan ditambahkan di akhir (addLast)

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1.	<pre>public Barang(String idBarang, String namaBarang, String kategori, int stok)</pre>	Constructor	Untuk menginisialisasi objek pada barang baru dengan mengisi keempat atribut tersebut sekaligus saat objek pertama kali dibuat
2	<pre>public void tambahBarangBaru(String idB, String namaB, String kategoriB, int StokB)</pre>	Procedural	Untuk membuat objek barang baru dengan memasukkan ke dalam map, memasukkan kategorinya ke set dan mencatat aktivitasnya me riwayat

	<pre>public void tambahkanStok(String idB, int jumlahB)</pre>	Procedural	Menambah nilai stok pada objek Barang didalam Map, jika ID ditemukan maka dicatat aktivitas ke riwayat
	<pre>public void kurangiStok(String idB, int jumlahB)</pre>	Procedural	Mengurangi stok barang jika ID ditemukan dan stok mencukupi, namun akan otomatis tertolak jika stok kurang atau tidak ada dari jumlah yang diminta
	<pre>public void cetakLaporan()</pre>	Procedural	Menampilkan laporan akhir data berisi daftar kategori dari Set, sisa stok semua barang dari Map dan urutan riwayat transaksi dari List.

IV. Dokumentasi dan Pembahasan Code

A. Barang.java

```
//nomor satu membuat class barang dengan atribut
public class Barang {
    String idBarang;
    String namaBarang;
    String kategori;
    int stok;

    //Konstruktor untuk atribut
    public Barang(String idBarang, String namaBarang, String
kategori, int stok) {
        this.idBarang = idBarang;
        this.namaBarang = namaBarang;
        this.kategori = kategori;
        this.stok = stok;
    }
}
```

```
}
```

- Sesuai dengan soal yang diberikan, class Barang berfungsi sebagai model utama yang mampu merepresentasikan satu barang yang ada di gudang. Keempat atribut tersebut dibungkus dalam satu class.
- Constructor digunakan untuk menginisialisasi seluruh atribut sekaligus saat objek baru saja dibuat. Jika dilihat dalam code terdapat kata kunci this, itu digunakan untuk membedakan atribut class dengan parameter yang mempunyai nama yang sama.

B. SistemGudang.java

```
public class SistemGudang { //class ssistem gudang
    Map<String, Barang> databaseBarang = new HashMap<>();
    Set<String> kategoriUnik = new HashSet<>();
    LinkedList<String> riwayat = new LinkedList<>();
```

- databaseBarang menggunakan HashMap sesuai soal sebagai database untuk Barang dengan Key yang merupakan idBarang, pencarian barang saat update stok sangat cepat karena HashMap bekerja dengan kompleksitas $O(1)$.
- kategoriUnik menggunakan HashSet sesuai dengan soal sebagai kategori unik. Set mampu memastikan tidak ada nama kategori yang ganda, jika ada kategori yang sama dimasukkan dua kali maka akan otomatis gagal atau diabaikan.
- riwayat menggunakan LinkedList sesuai dengan soal sebagai pencatat riwayat pada aktivitas setiap String aktivitas seperti misalnya Barang masuk dengan ID BAR01 ditambah 5 unit. Maka akan otomatis ditambahkan ke akhir list menggunakan fungsi addLast().

```
    public void tambahBarangBaru(String idB, String namaB,
String kategoriB, int StokB) {
        //buat objek dari Class Barang.java yang sebelumnya
sudah dibuat dengan konstruktor
        Barang barang = new
Barang(idB, namaB, kategoriB, StokB);
        //masukkan objek barang ke HashMap dengan idB
sebagai key nya
        databaseBarang.put(idB, barang);
        //masukkan kategoriB ke kategoriUnik (hash set) jika
barang sudah ada maka hashset akan otomatis menolak
        kategoriUnik.add(kategoriB);
        //menambahkan atau mencatat riwayat pendaftaran
barang baru ke paling akhir (addLast)
        riwayat.addLast("Barang Baru: " + idB + " - " +
namaB + " [Stok Awal: " + StokB + "]\n");
        System.out.println(" [SIP] Barang baru sudah
didaftarkan dengan Nama barang: " + namaB);
    }
```

- Pada method ini berfungsi untuk memasukkan objek barang ke Map dan kategorinya ke set, new Barang() membuat objek baru dalam class Barang menggunakan konstruktor lalu disimpan lagi ke dalam barang.
- databaseBarang.put(idB, barang) memasukkan objek tersebut ke HashMap dengan id sebagai keynya.
- kategoriUnik.add(kategoriB) memasukkan kategori ke HashSet. Jika sudah ada, otomatis diabaikan tanpa error.
- riwayat.addLast() mencatat aktivitas di posisi paling akhir LinkedList agar urutan kronologi terjaga.

```
public void tambahkanStok(String idB, int jumlahB){
    //mengecek apakah idB ada di dalam database sebagai
    key pada HashMap
    if (databaseBarang.containsKey(idB)) {
        //jika ada maka lakukan penambahan stok
        //get mengambil objek Barang dari HashMap
        berdasarkan key nya lalu stok-nya akan langsung ditambahkan
        dengan jumlahB
        Barang barang = databaseBarang.get(idB);
        barang.stok += jumlahB;
        //biasa, mencatat aktivitas ke riwayat dan
        menampilkan konfirmasi ke layar
        riwayat.addLast("Barang masuk: " + idB + " -
        ditambahkan stok sebanyak: " + jumlahB + " unit");
        System.out.println(" [SIP] Stok barang: " + idB
        + " berhasil ditambahkan dengan jumlah: " + jumlahB);
        //jika idB tidak ditemukan maka gagal
    } else {
        System.out.println(" [GAGAL] Barang dengan ID "
        + idB + " tidak ditemukan.");
    }
}
```

- Method ini berfungsi untuk menambahkan stok pada objek didalam Map jika ID ditemukan, maka containsKey(idB) akan mengecek apakah ID ada sebagai key di HashMap atau tidak. Validasi ini wajib dilakukan sebelum mengubah stok.
- databaseBarang.get(idB) mengambil objek pada Barang dari HashMap berdasarkan key, lalu akan langsung ditambahkan stoknya. Jika ID tak ada maka akan masuk ke else dan menampilkan pesan gagal.

```
public void kurangiStok(String idB, int jumlahB){
    //tolak jika idB tidak ada atau tidak ditemukan
    if (!databaseBarang.containsKey(idB)) {
```

```

        System.out.println(" [GAGAL] Barang dengan ID " +
idB + " tidak ditemukan.");
        return;
    }
    Barang barang = databaseBarang.get(idB);
    //mengurangi stok jika stok cukup
    if(barang.stok >= jumlahB) {
        barang.stok -= jumlahB;
        riwayat.addLast("Barang keluar: " + idB + " -
dikurangi stok sebanyak: " + jumlahB + " unit");
        System.out.println(" [SIP] Stok barang: " + idB
+ " berhasil dikurangi dengan jumlah: " + jumlahB);
    } else {
        //tolak karena stok tidak mencukupi atau stok
kurang
        System.out.println(" [GAGAL] Stok barang: " +
idB + " kurang dengan jumlah: " + jumlahB);
    }
}

```

- Method ini berfungsi untuk mengurangi stok dengan syarat bahwa stok harus cukup, jika tidak maka akan gagal atau ditolak.
- Pada method ini memiliki dua validasi, pertama mengecek keberadaan ID dengan containsKey(idB). Tanpa ! yang artinya jika tidak ditemukan maka ID tidak ada dan langsung menampilkan pesan gagal dan menghentikan method dengan return.
- Validasi kedua mengecek kecukupan stok dengan barang.stok >= jumlah, jika stok mencukupi maka akan dikurangi jika tidak maka akan di tolak, alasan kegagalan tetap dicatat dalam riwayat.

```

public void cetakLaporan() {
    System.out.println("=====");
    System.out.println("=== Laporan Gudang ===");
    System.out.println("=== Daftar Kategori Barang
===");

    for (String kategori : kategoriUnik) { //untuk
setruap kategori dnegan tipe string yang ada pada kategori
unik maka lakukan perulangan
        System.out.println("- " + kategori);
    }
}

```

```

        System.out.println("=== Sisa Stok Semua Barang
==="); //mengubah seluruh isi hashmap menjadi kumpulan yang
bisa diiteraskan
        for (Map.Entry<String, Barang> entry :
databaseBarang.entrySet()) { //dengan tipe data yang sesuai
dengan key dan value
            Barang b = entry.getValue();
            System.out.println("  [" + b.idBarang + "] " +
b.namaBarang + " | Kategori: " + b.kategori + " | Stok: " +
b.stok);
        }

        System.out.println("=== Riwayat Transaksi ===");
        for (String transaksi : riwayat) {
            System.out.println("- " + transaksi);
        }

System.out.println("=====");
    }

```

- Method ini berfungsi untuk mencetak laporan yang sudah dilakukan sebelumnya.
- for (String kategori : kategoriUnik) merupakan for each loop, titik dua (:) dibaca "untuk setiap", artinya iterasi setiap elemen String yang ada di dalam HashSet kategoriUnik.
- databaseBarang.entrySet() mengubah seluruh isi HashMap menjadi kumpulan pasangan Key-Value yang bisa diiterasi. Map.Entry<String, Barang> adalah tipe data satu pasang Key-Value tersebut. entry.getValue() mengambil bagian Value-nya saja yaitu objek Barang.
- for (String transaksi : riwayat) mengiterasi seluruh isi LinkedList dari elemen pertama hingga terakhir. Urutan dijamin sesuai kronologi karena LinkedList menjaga urutan addLast().

C. Main.java

```

public class Main {
    public static void main(String[] args) {
        //buat objek dari sistemgudang
        SistemGudang gudangs = new SistemGudang();

        //Daftarkan minimal 3 barang baru.
        gudangs.tambahBarangBaru("BAR01", "Beras 10kg",
"Sembako", 100);
    }
}

```

```

        gudangs.tambahBarangBaru("BAR02", "Minyak Goreng
1L", "Sembako", 100);
        gudangs.tambahBarangBaru("BAR03", "Air Puti Mineral
500ml", "Minuman", 50);

        //Lakukan 1x tambah stok yang berhasil.
        gudangs.tambahkanStok("BAR01", 5);

        //Lakukan 1x kurangi stok yang berhasil
        gudangs.kurangiStok("BAR02", 10);

        //Lakukan 1x kurangi stok yang GAGAL (karena stok
ditarik lebih besar dari sisa).
        gudangs.kurangiStok("BAR03", 100);

        //Panggil metode cetak laporan akhir.
        gudangs.cetakLaporan();
    }
}

```

- SistemGudang gudangs = new SistemGudang(); Membuat satu objek bernama gudangs dari class SistemGudang. Seluruh data barang, kategori, dan riwayat transaksi akan tersimpan di dalam objek ini selama program berjalan.
- gudangs.tambahBarangBaru("BAR01", "Beras 10kg", "Sembako", 100);
gudangs.tambahBarangBaru("BAR02", "Minyak Goreng 1L", "Sembako", 100);
gudangs.tambahBarangBaru("BAR03", "Air Putih Mineral 500ml", "Minuman", 50); Artinya ketiga barang didaftarkan ke dalam sistem dengan memanggil tambahBarangsBaru() sebanyak tiga kali.
- gudangs.tambahkanStok("BAR01", 5); Menambahkan stok dan berhasil.
- gudangs.kurangiStok("BAR02", 10); Mengurangi stok dan berhasil
- gudangs.kurangiStok("BAR03", 100); Mengurangi stok namun gagal karena unit yang tersedia tidak cukup.
- gudangs.cetakLaporan(); Menampilkan tiga bagian laporan sekaligus: daftar kategori unik dari HashSet, sisa stok semua barang dari HashMap, dan seluruh riwayat transaksi dari LinkedList secara berurutan sesuai kronologi.

D. Output dari Program

```

[SIP] Barang baru sudah didaftarkan dengan Nama barang: Beras 10kg
[SIP] Barang baru sudah didaftarkan dengan Nama barang: Minyak Goreng 1L
[SIP] Barang baru sudah didaftarkan dengan Nama barang: Air Puti Mineral
500ml
[SIP] Stok barang: BAR01 berhasil ditambahkan dengan jumlah: 5
[SIP] Stok barang: BAR02 berhasil dikurangi dengan jumlah: 10
[GAGAL] Stok barang: BAR03 kurang dengan jumlah: 100
=====
=== Laporan Gudang ===
=== Daftar Kategori Barang ===
- Sembako
- Minuman
=== Sisa Stok Semua Barang ===
[BAR01] Beras 10kg | Kategori: Sembako | Stok: 105
[BAR02] Minyak Goreng 1L | Kategori: Sembako | Stok: 90
[BAR03] Air Puti Mineral 500ml | Kategori: Minuman | Stok: 50
=== Riwayat Transaksi ===
- Barang Baru: BAR01 - Beras 10kg [Stok Awal: 100]
- Barang Baru: BAR02 - Minyak Goreng 1L [Stok Awal: 100]
- Barang Baru: BAR03 - Air Puti Mineral 500ml [Stok Awal: 50]
- Barang masuk: BAR01 - ditambahkan stok sebanyak: 5 unit
- Barang keluar: BAR02 - dikurangi stok sebanyak: 10 unit
=====

```

V. Kesimpulan

Berdasarkan hasil implementasi dan uji program simulasi sistem ini pada THT 10, dapat disimpulkan bahwa Java Collections menyediakan struktur data yang lebih fleksibel dibandingkan array biasa karena ukurannya dinamis. Pemilihan collection yang tepat sangat penting, di mana HashMap terbukti efisien sebagai database in-memory, HashSet efektif mengelola data unik tanpa pengecekan manual, dan LinkedList cocok untuk pencatatan log yang berurutan. Selain itu, penerapan dua lapis validasi pada method kurangiStok() membuktikan bahwa program mampu menolak transaksi yang tidak valid, sehingga seluruh aktivitas tercatat secara kronologis dan laporan akhir menampilkan informasi yang lengkap dan akurat.

VI. Daftar Pustaka

-